

Real-Time Fraud Detection & Prevention in Digital Lending Ecosystems

AI-Powered Detection · Behavioral Analytics · Explainable Decisioning

A transaction-type-aware pipeline that classifies each event, activates only the relevant fraud signals, blends deterministic ML with vector similarity, and returns an explainable real-time decision.

TEAM

Abhinav Pal

2022B2A11556G · BITS Pilani, Pilani Campus

HACKATHON

Synchrony Hackathon
Problem Statement PS-2

DATE

September 2026

Source code: github.com/abhinav-pal246/Synchrony_Real_Fraud_Detection_and_Prevention_In_digital_Landing

THE PROBLEM

Static rules cannot keep pace with evolving fraud

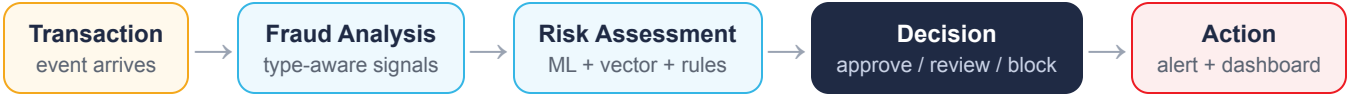
Why traditional detection breaks down

- Digital lending drives **high-volume, low-latency** transaction streams.
- Fraud patterns evolve continuously — rules lag behind **novel** attacks.
- Applying **one model / one feature-set to every transaction** is inefficient and blind to context.
- Aggressive rules raise **false positives** that block genuine customers.
- Analysts get a "fraud" flag with **no reason** — no basis for a fast, defensible decision.

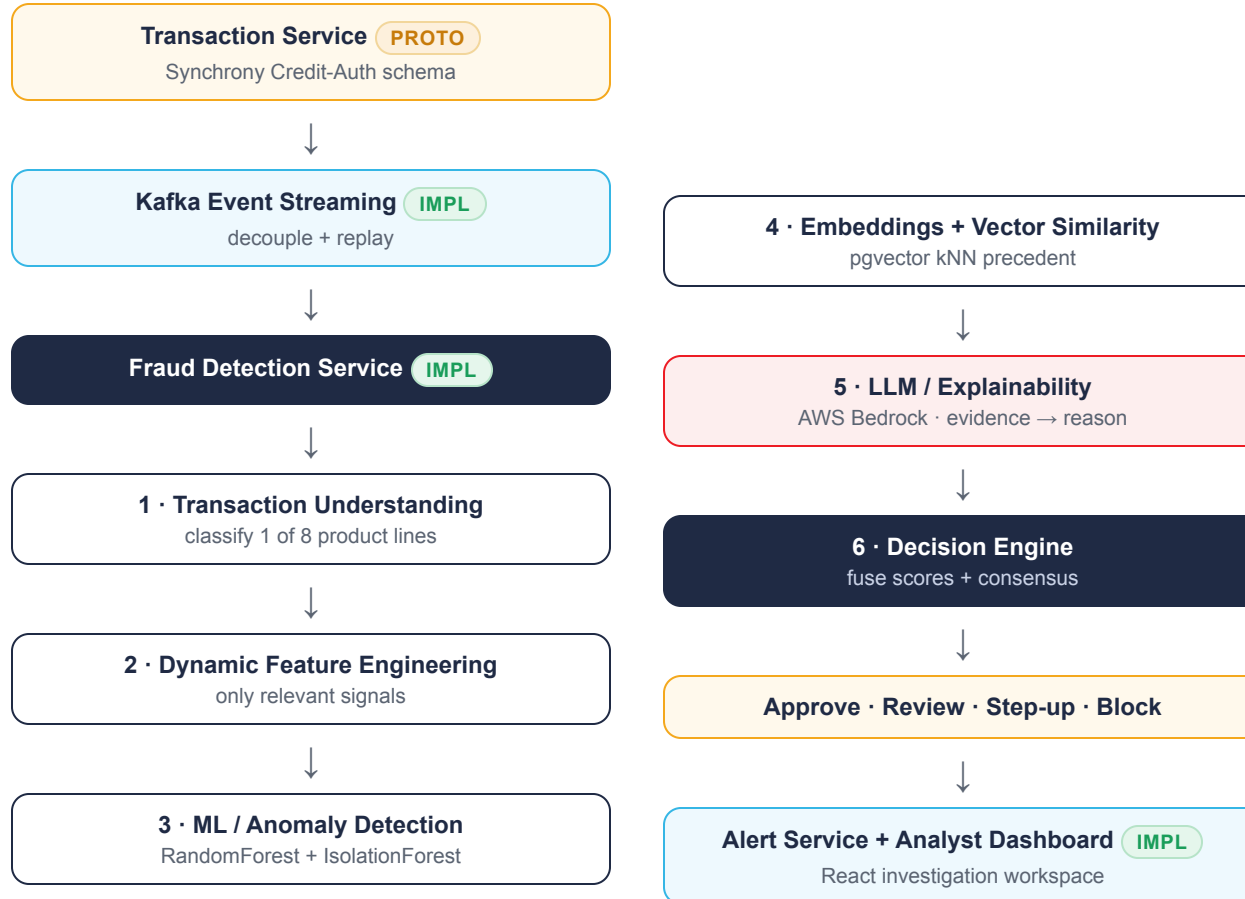
The core challenge

“How can we dynamically analyze every transaction, identify the appropriate fraud signals, detect known and unknown patterns, and make an explainable real-time decision?”

Our answer: make the pipeline **transaction-type-aware** and **evidence-first** — the system explains what it saw, not just what it concluded.



A layered, real-time fraud-detection platform



Design principle

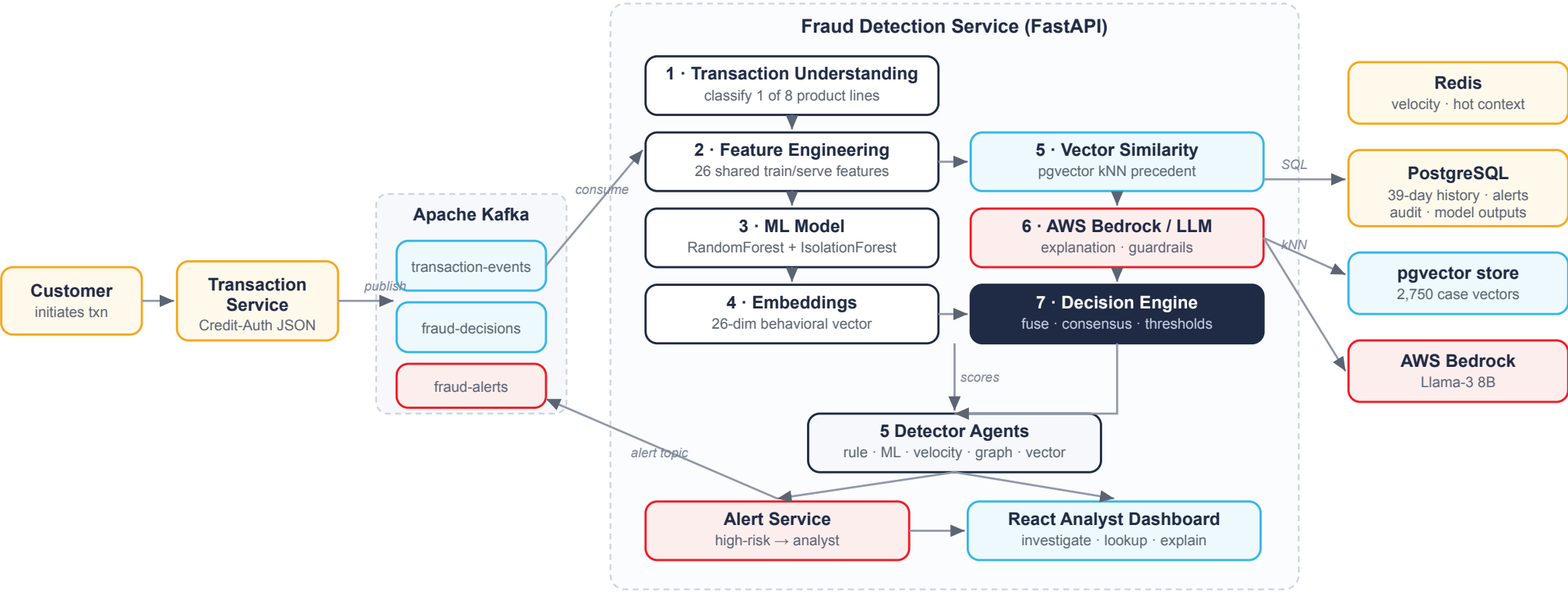
The platform **does not blindly apply one model to every transaction**. Each event is first **classified**, which activates only the fraud vectors that can actually occur for that product line.

Deterministic models own the **risk score**; the LLM only **explains**. The Decision Engine — not the LLM — makes the call.

Every layer exists to answer a specific question: *what is this?, is it anomalous?, have we seen it before?, why?, and what do we do?*

ARCHITECTURE

End-to-end system architecture



What happens when a transaction arrives

#	Step	Detail
1–3	Ingest & publish	Transaction Service emits the Credit-Auth event → Kafka transaction-events
4	Consume	Fraud Detection Service consumes the event
5	Classify	Identify the product line / transaction type (1 of 8)
6	Retrieve context	Pull the account's 39-day history from PostgreSQL + hot velocity from Redis
7	Feature engineering	Build the 26-feature vector (amount, z-score, channel, CVV/AVS, MCC...)
8	ML scoring	RandomForest probability + IsolationForest anomaly
9–10	Vector search	Embed → pgvector kNN over 2,750 labelled cases → nearest precedents
11	Explain	LLM receives structured evidence → human-readable reason
12–13	Decide	Decision Engine fuses score + rules + similarity → APPROVE / REVIEW / BLOCK
14–15	Act	Alert Service notifies analyst; dashboard shows txn + explanation

Example event travelling the pipeline

```

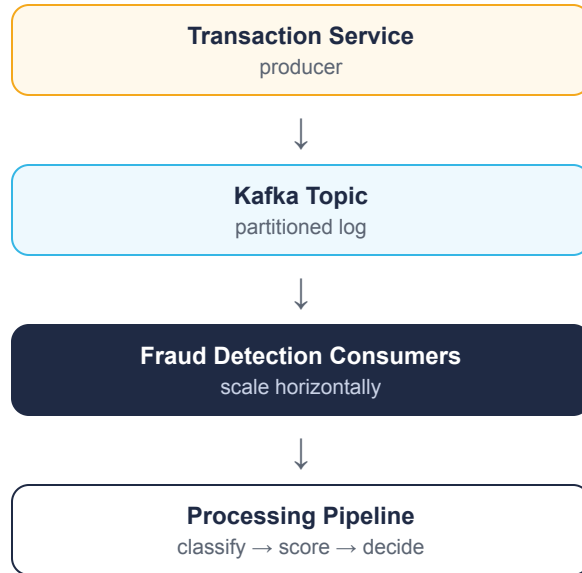
{
  "account_id": "ACC-000468",
  "amount": 0.75,           // micro-charge
  "channel": "e_commerce",
  "entry_method": "cnp",
  "cvv_result": "no_match",
  "avs_result": "N",
  "status": "declined"
}

```

Pipeline verdict LIVE OUTPUT

Type: Co-branded e-commerce · **Vector:** **CB-2** card-testing
Signals: cvv_mismatch · avs_mismatch · small_amount_decline
Nearest cases: 5 / 8 labelled fraud (CB-2)
Decision: **BLOCK / step-up** · risk 1.00

Why Kafka? — decoupling and replay



What Kafka buys us

- **Decouples** transaction processing from fraud detection.
- High-throughput, **asynchronous** event streaming.
- Multiple independent **consumers** (scoring, audit, analytics).
- **Replay** — re-run history against a new model version.
- Producer is never coupled to, or blocked by, the fraud engine.

Topic organisation

transaction-events raw authorisation events (producer → detection)

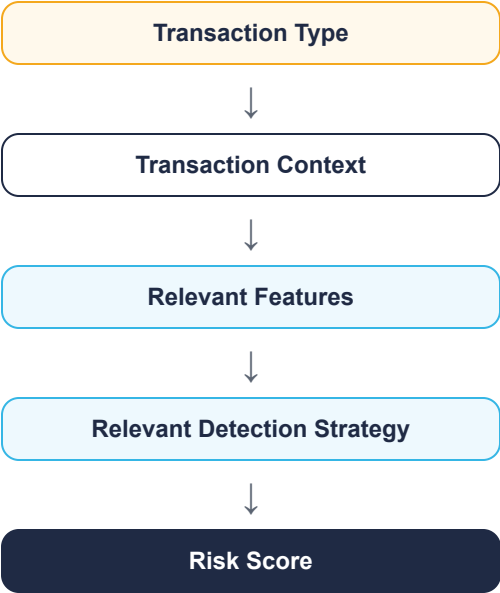
fraud-decisions scored results for downstream systems

fraud-alerts high-risk cases → Alert Service → dashboard

IMPL Kafka (KRaft) runs in the Docker stack. **PROTO** live consumer wiring is partial — detection currently runs synchronously via the API for the demo.

CORE IDEA

Dynamic, transaction-type-aware detection



Instead of one model for everything, classification **activates only the fraud vectors that can occur** for that product line.

Transaction type	Illustrative relevant signals
Private-label credit card	gift-card MCC, promo abuse, in-store entry mode
Co-branded / dual card	CVV/AVS mismatch, magstripe fallback, card-testing bursts
Promotional financing	promo-plan misuse, amount thresholds, velocity
POS installment loan	identity signals, first-payment default risk
CareCredit transaction	health-MCC pattern, provider anomalies
Real-time credit application	identity + application velocity, device/synthetic-ID history
Digital wallet payment	token provisioning age, device change, account-takeover
E-commerce checkout	CNP, shipping/billing mismatch, IP + device fingerprint

Payment txn → signals

amount · merchant · device · location · velocity · historical behaviour

Credit application → signals

identity · application velocity · device history · account history · behavioural anomalies

IMPL 8-type taxonomy + 11 observed fraud vectors in the dataset; rule-map + ML select the active signals. **PROPOSED** per-type specialist sub-models.

READING THE CODES

Fraud-vector & actor taxonomy

Every alert carries a **VECTOR** (what pattern) and an actor (who did it). Format: <product-line>-<n>. The counts below are the fraud cases actually present in our 40-day dataset.

Vector	Meaning	Cases
CB-2	Co-branded card-testing — bursts of tiny declined CNP auths (stolen-card validation)	77
PF-1	Promotional-financing abuse — exploiting 0%/deferred-interest plans	40
CB-4	Co-branded account misuse / unauthorized high-value spend	26
APP-2	Real-time credit application fraud — synthetic-identity onboarding	25
PL-2	Private-label gift-card / stored-value cash-out purchase	24
CC-1	CareCredit anomaly — provider / health-MCC amount irregularity	22
X-1	Cross-cutting high-amount anomaly (applies across product lines)	12
EC-1	E-commerce CNP with CVV/AVS mismatch	10
DW-1	Digital-wallet account-takeover / suspicious token provisioning	6
CB-1	Co-branded magstripe fallback / counterfeit entry	4
CB-3	Co-branded cash-like MCC abuse (quasi-cash / money transfer)	4

Product-line prefixes

PL Private-label · CB Co-branded / dual · PF Promotional financing · IL POS installment · CC CareCredit · APP Credit application · DW Digital wallet · EC E-commerce · X Cross-cutting

Actor codes — who initiated

Code	Actor	Cases
3P	Third-party fraudster (external)	109
B	First-party / cardholder (friendly fraud)	64
M	Merchant / associate (internal)	48
S	Synthetic identity / organized ring	29

Signals grouped by what they capture

Customer

- Transaction frequency
- Average / z-scored amount
- Historical spending profile
- Account age
- Prior fraud alerts

Transaction

- Amount
- Timestamp / hour
- Payment & entry method
- Transaction type
- Merchant · MCC · channel

Behavioral

- Velocity
- Sudden spend change
- Unusual timing
- Location deviation
- Device change

Network / relationship

- Shared device
- Shared IP
- Shared merchant
- Repeated suspicious entities
- Transaction linkage

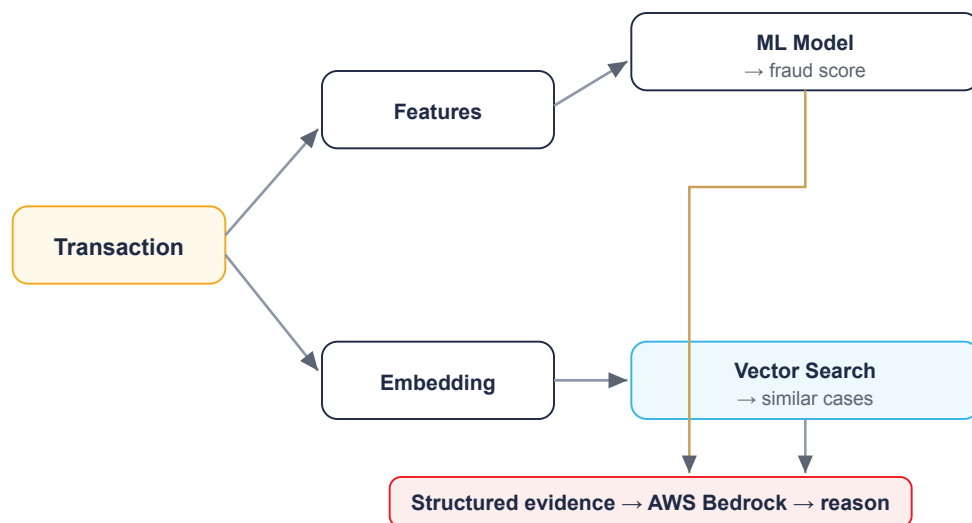
Behavioral baseline from 39-day history IMPL

Each account's prior 39 days are stored in PostgreSQL. On the 40th ("current") day we retrieve **only that account's** history (index-backed) to compute its baseline — mean/spend profile, channel mix, first/last seen — so anomalies are judged **relative to the customer**, not a global average.

26 shared features · one function

```
FEATURE_ORDER = [  
  "amount","hour","amount_zscore",  
  "cvv_no_match","avs_no_match",  
  "acct_txn_count","acct_avg_amount",  
  chan_*, entry_*, mcc_* ... ] # train==serve
```

ML + Embeddings + Vector search — distinct roles



ML model IMPL

RandomForest (fraud probability) + IsolationForest (anomaly). Produces the **core numerical risk**.

Embeddings IMPL

Scaled 26-dim behavioral vector — a transaction's semantic/behavioral fingerprint.

pgvector IMPL

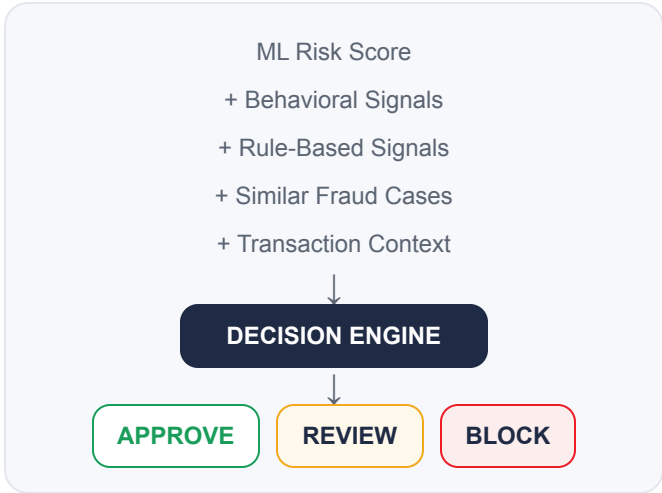
kNN over **2,750** labelled cases (250 fraud + 2,500 legit) → nearest historical precedents + RAG.

LLM is not the detector

Deterministic models produce the risk. The LLM only does **contextual reasoning, explanation and analyst assistance**.

DECISION ENGINE

Controlled fusion — the model decides, not the LLM



Thresholds (implemented): risk ≥ 0.85 → decline · ≥ 0.60 → step-up · ≥ 0.40 → manual review · else approve. A **consensus boost** raises risk when multiple agents agree.

Guardrail: no uncontrolled financial decisions

The LLM never issues an approve/block. It returns text; the deterministic **Decision Engine** maps fused risk → action. This keeps the money-path auditable and reproducible.

Every decision is persisted for audit IMPL

- Risk score
 - Triggered signals
 - Relevant rules / vectors
 - Similar historical cases
 - Decision + actor
 - Explanation
 - Timestamp
 - Model / version

Evidence, not just a "fraud" flag

Transaction

\$2,850

unseen-device purchase



RISK: HIGH

Final decision → **REVIEW / BLOCK**

This is an **illustrative** explanation format; the live system emits the same evidence structure (see dashboard).

Why this was flagged — ranked evidence

1. Amount is **significantly above** the customer's historical average.
2. Originated from a **previously unseen device**.
3. **Multiple transactions** occurred within a short time window (velocity).
4. **Location differs** from recent customer activity.
5. Similar historical transactions were **associated with fraud** (vector precedent).

Live evidence contract IMPL

The `/investigate` and `/lookup` endpoints return: indicators, contributing signals, nearest labelled cases, ML score, actor + fraud vector, and a Bedrock-generated reason — surfaced directly in the analyst UI.

Fraud investigation workspace

IMPLEMENTED

synchrony

ANALYTICS

• Live · Postgres + pgvector + Redis

Overview

Fraudulent Accounts

Account Lookup

52,260

Transactions

85

Fraud accts

250

Alerts

188

Critical

CB-2

Top vector

Detected Fraudulent Accounts

ACC-000468	CB-2	24	100	\$598.59
ACC-000166	CB-2	22	100	\$23.25
ACC-000595	CB-2	17	100	\$15.00
ACC-000314	X-1	1	99	\$1,715

Fraud Account Lookup

Account ID

Txn ID

ACC-000468

FRAUDULENT · CB-2 · third-party

cvv mismatch · avs mismatch · small-amt

decline

Live React + Tailwind dashboard (rendered layout shown), backed by real API calls over the PostgreSQL history store.

Top metrics

transactions analyzed · fraudulent accounts · fraud alerts · critical alerts · top fraud vector (blocked / under-review counts & avg latency — **PLANNED**)

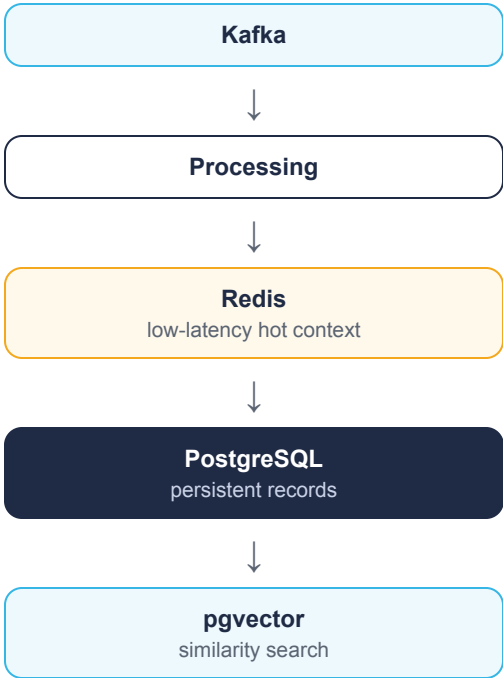
Fraudulent account list → detail

Click any account to see: identifier · triggering transactions · who initiated (actor) · why flagged · indicators · ML score · 39-day context · recent history.

Fraud Lookup

Search by **primary key** — account_id or transaction_id — returns fraud status + evidence. Customer ID / merchant ID keys — **PLANNED**.

Why Redis and PostgreSQL + pgvector



PostgreSQL

IMPL

- Customer + account data
- Transaction records (39-day history)
- Fraud decisions / alerts
- Model outputs
- Audit trail

1,000 accounts · 52,260 txns loaded · indexed (account_id, event_time).

pgvector

IMPL

- Transaction embeddings
- Historical fraud embeddings
- kNN similarity search

2,750 labelled case vectors (26-dim).

Redis

PROTO

- Hot customer context
- Recent-transaction features
- Velocity counters
- Feature caching

Running in stack; velocity agent integration in progress.

Efficient context retrieval — never dump the whole DB to the model

On the 40th day we fetch **one account's** prior-39-day rows via the composite index, compute its baseline in-service, and pass only that context to feature-engineering + embeddings. This keeps scoring fast and privacy-scoped.

Security · Responsible AI · Engineering

Security

- JWT auth, bcrypt-hashed credentials **IMPL**
- No hardcoded login — verified in DB **IMPL**
- Pydantic input validation **IMPL**
- Scoped DB access, CORS **IMPL**
- RBAC / roles **PROPOSED**
- Secrets manager, TLS, at-rest encryption **PROPOSED**

Responsible AI

- Explainable decisions (evidence) **IMPL**
- Human-in-the-loop review/step-up **IMPL**
- LLM guardrails + PII redaction **IMPL**
- No unrestricted LLM decisioning **IMPL**
- Model / bias monitoring **PROPOSED**
- Full auditability of decisions **PROTO**

Engineering

- Modular services, API-first **IMPL**
- Docker Compose stack **IMPL**
- Alembic migrations, versioned models **IMPL**
- Git / GitHub workflow **IMPL**
- Structured logging **PROTO**
- Unit tests, CI, monitoring **PROPOSED**

Honesty note for evaluators

Constraints are stated, not hidden: the AWS Organization SCP blocks Secrets Manager / Cognito, so credentials are bcrypt-hashed in PostgreSQL for now; Anthropic Claude needs account verification, so the LLM layer runs on **Bedrock Llama-3 8B**. These are deployment choices, clearly labelled.

Prototype results and the path to production

52,260

transactions loaded & scorable

~15 ms

ML + rule scoring / txn (measured)

~3.1 s

full /investigate incl. Bedrock LLM (measured)

2,750

pgvector labelled cases

How it scales

- Kafka horizontal scaling · multiple consumers
- Stateless FastAPI services
- Redis caching · PostgreSQL indexing
- pgvector ANN similarity
- AWS deployment · model versioning · observability

Future roadmap

- Online / incremental learning
- Graph-based fraud-ring detection
- Advanced behavioral profiling
- Agent-assisted investigation
- Continuous monitoring + automated feedback loop
- Cross-merchant fraud intelligence

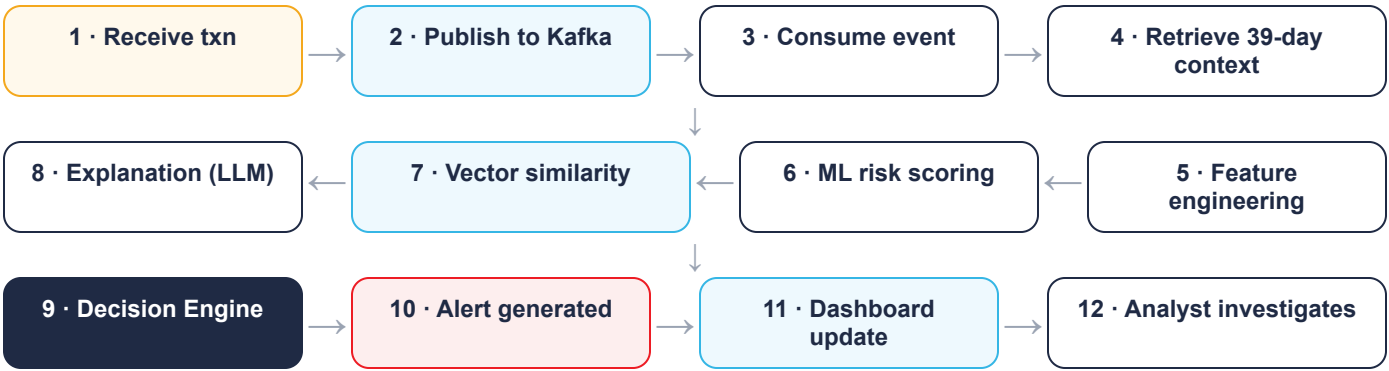
Prototype evaluation **IN-SAMPLE**

Metric (risk ≥ 0.50)	Value	Metric	Value
Recall (245 / 250 frauds)	0.98	Precision	1.00
F1	0.99	False-positive rate	0.000

Read these critically: the models were trained on this **synthetic** dataset with no held-out split, and the injected fraud carries clearly-encoded signals — so these are **in-sample** figures that confirm the pipeline learns the intended patterns, **not** generalization estimates. A stratified train/test split, threshold tuning, and evaluation on harder / adversarial data are the immediate next steps.

DEMO FLOW

From event to explainable decision



“From transaction event → contextual intelligence → explainable risk → real-time decision.”